

Week 8 portfolio last assignment design

#CS 255 Portfolio: System Analysis & Design (FrivverPass Project) Kamaljit Rooprai

This repo contains my work and showing my process for collecting requirements from a client and designing a system that really meets their needs.

#What's in here

****Business Requirements Document****(Project 1)

CS 255 Business Requirements Document Template

System Components and Design

Purpose

What is the purpose of this project? Who is the client and what do they want their system to be able to do?

- This project is for our client, DriverPass. They noticed that a lot of people fail their driving exam, so they want to build a company that helps people get ready for it.
- DriverPass wants a system that lets customers take online classes, do practice tests, and book real on the road lessons with an actual driver.
- Generally, they need us to build the software that runs their whole training business, from booking a lesson to tracking a customer's progress.

System Background

What does DriverPass want the system to do? What is the problem they want to fix? What are the different components needed for this system?

- Customers need to be able to book a two-hour driving lesson online via their account, or by calling or visiting the office and speaking with the secretary.
- DriverPass has several cars and multiple drivers, therefore the system needs to match each booked lesson with an available car, driver, and time slot.
- Customers choose from 3 packages that has different amounts of driving time, an in-person class about DMV rules, or access to online course with practice exams.
- Liam wants to see an activity report, so he always knows who made, canceled, or last changed a reservation, in case something goes wrong.
- The system needs to stay connected to the DMV so DriverPass always has the latest rules, policies, and practice questions.

Objectives and Goals

What should this system be able to do when it is completed? What measurable tasks need to be included in the system design to achieve this?

- Let customers create an account and log in with their shipping, billing, and contact information.
- Let customers book, change, or cancel a driving lesson virtually.
- Match each booked lesson with the correct driver, car, and time spot.
- Let customers pick one of the 3 training packages.

- Track a customer's online test progress.
- Give Liam and Ian a simple way to see who did what on a reservation.
- Get updated automatically whenever the DMV updates its rules or practice questions.

Requirements

Nonfunctional Requirements

Performance Requirements

What environments (web-based, application, etc.) does this system need to run in? How fast should the system run? How often should the system be updated?

- The system needs to be run as web app that works from any computer or mobile device, since customers should be able to book lessons from wherever they are.
- Liam should be able to view this data virtually or offline, but he can only make changes to it while he is online. It keeps the data from getting out of sync throughout several servers.
- Liam should be able to download reports and work through them at home by using a program like Excel.

Platform Constraints

What platforms (Windows, Unix, etc.) should the system run on? Does the back end require any tools, such as a database, to support this application?

- This system should run on the web, essentially hosted in the cloud, since DriverPass does not want to deal with their own servers, backups as well as security.
- DriverPass wants to focus on running their business, not on fixing technical problems, so the hosting provider should handle backups and security.

Accuracy and Precision

How will you distinguish between different users? Is the input case-sensitive? When should the system inform the admin of a problem?

- The system needs several roles for different people such as: Liam (owner, full visibility), Ian (IT, full access so he can reset or block accounts), the secretary (can book appointments for customers), and customers (can only manage their own bookings).
- The system should log who made, canceled, or last modified every reservation, so Liam can figure out who is responsible if there is an error.

Adaptability

Can you make changes to the user (add/remove/modify) without changing code? How will the system adapt to platform updates? What type of access does the IT admin need?

- Liam wants to be able to turn off a package so new customers can't sign up for it, without needing a developer to do it.
- Also, adding or removing a whole new package is a larger change and will still require a developer, so that can wait for a future update.

- The system should be able to receive notifications from the DMV on new rules, regulations, or practice questions without needing a full system redesign.

Security

What is required for the user to log in? How can you secure the connection or the data exchange between the client and the server? What should happen to the account if there is a “brute force” hacking attempt? What happens if the user forgets their password?

- Ian requires full admin access so he can reset a customer’s/employee’s password, or block someone’s access if they are let go.
- Customers should be able to reset their own password automatically if they forget it.
- Payment information like card digits, expiration date, security code, or pin needs to be captured and handled securely.

Functional Requirements

Using the information from the scenario, think about the different functions the system needs to provide. Each of your bullets should start with “The system shall . . .” For example, one functional requirement might be, “The system shall validate user credentials when logging in.”

- The system shall let a customer register an account with their first name, last name, address, phone number, state, and payment information.
- The system shall let a customer book, reschedule, or cancel a driving lesson virtually.
- The system shall match each booked lesson with an available driver and car.
- The system shall let a customer choose one of the 3 available training packages.

- The system shall let the secretary create an appointment for a customer in person or on the phone.
- The system shall track who created, modified, or canceled each reservation.
- The system shall update the customer about their lesson and confirmation.
- The system shall alert DriverPass employee whenever a new transaction happens.
- The system shall let a customer view their order history and lesson status online at any given time.
- The system shall let Liam disable a package so no new customers can sign up for it.
- The system shall automatically reset a customer's password if they forget it.
- The system shall receive notifications from the DMV on the new rules or practice exam content.

User Interface

What are the needs of the interface? Who are the different users for this interface? What will each user need to be able to do through the interface? How will the user interact with the interface (mobile, browser, etc.)?

- Customers require a screen that shows their online test progress: the test name, how long it took, their score, and the status.
- Customers require to see a simple table of their driving lessons, showing the lesson time, start hour, end hour, and any notes their driver left.
- There needs to be an input from where a student or the secretary can fill in the student's name, address, phone number, state, and payment details.

- There shall be a contact page so customers can reach DriverPass, and a way for DriverPass to get the customer.
- The interface requires to work for customers booking lessons, the secretary managing appointments, and Ian handling admin tasks.

Assumptions

What things were not specifically addressed in your design above? What assumptions are you making in your design about the users or the technology they have?

- Customers have a working debit/credit card to pay for their package.
- Customers have internet access and a device to log into their account.
- DriverPass's office has consistent internet access so the system stays connected to the central system.

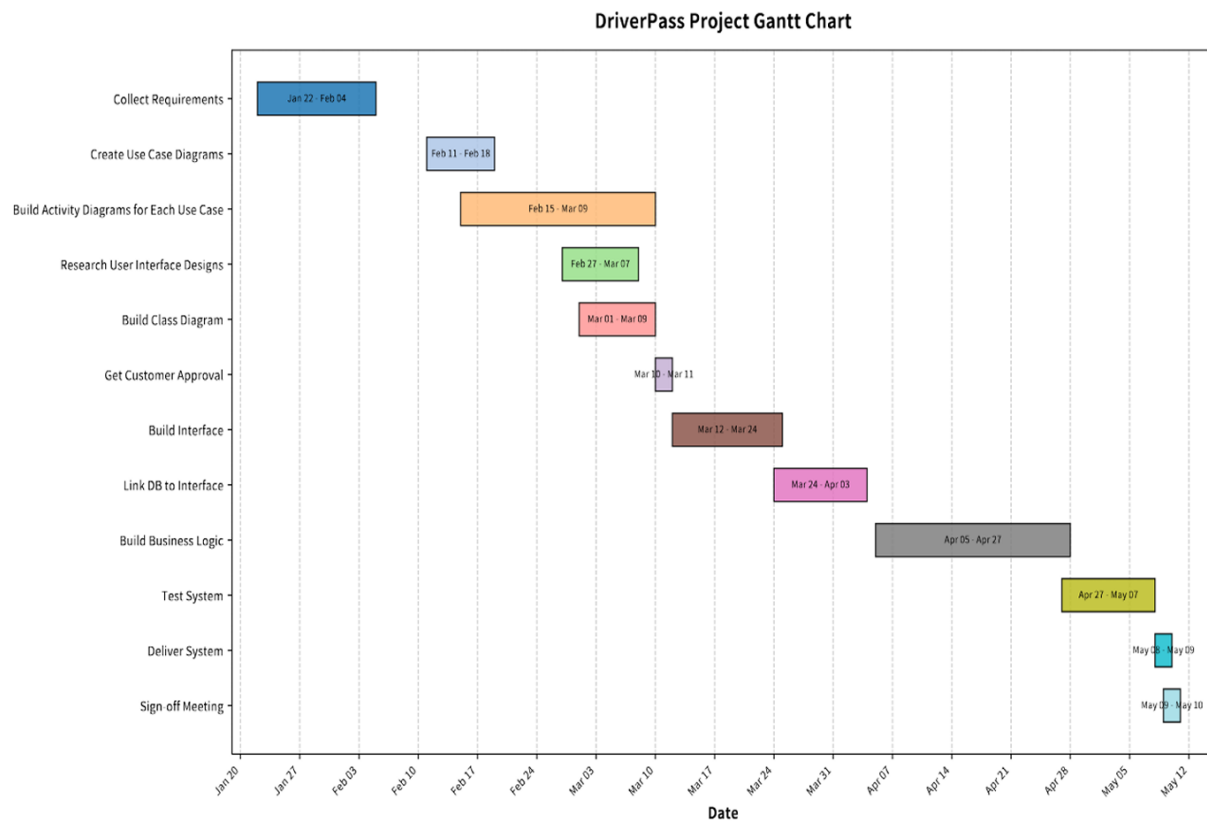
Limitations

Any system you build will naturally have limitations. What limitations do you see in your system design? What limitations do you have as far as resources, time, budget, or technology?

- The system would not be able to add or remove the whole new packages without a developer because that is something planned for a future release but not this version.
- The system relies on being online to save changes; nothing can be updated while offline.
- The system tracks down bookings and drivers but it does not include things like real time GPS tracking of the real world cars.

Gantt Chart

Please include a screenshot of the GANTT chart that you created with Lucidchart. Be sure to check that it meets the plan described by the characters in the interview.



****System Design Document**** (Project 2)

CS 255 System Design Document Template

Project Two – DriverPass System Design

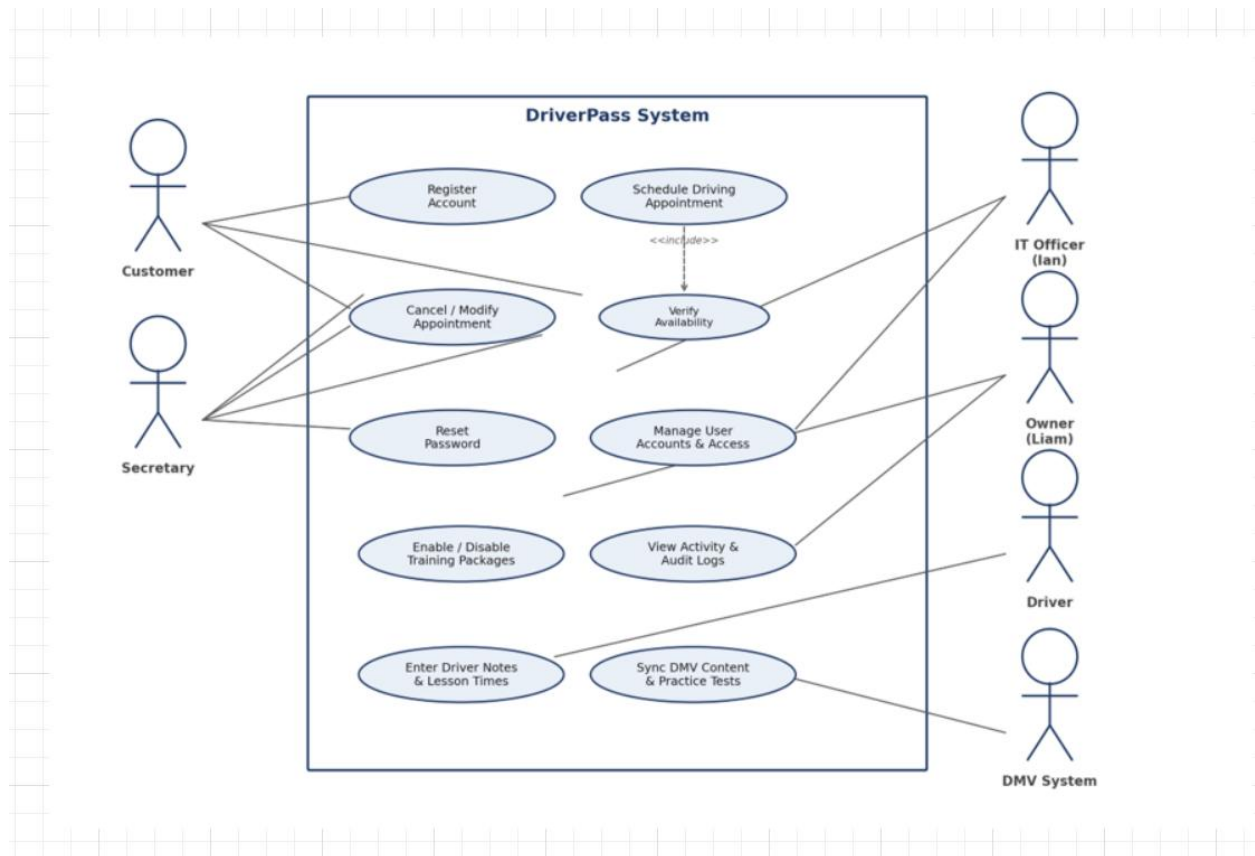
Kamaljit Rooprai

UML Diagrams

UML Use Case Diagram

This diagram below shows the different people and outside systems who use DriverPass, and what they can do. A customer can sign up, book a lesson, or cancel/change a booking on their own. The secretary can do those same booking tasks over the phone as well as in person for a customer who calls in. Ian, the IT officer, handles account resets and access. Liam, the owner, can turn training packages on or off and pull activity reports. Drivers log their notes and lesson times, and the DMV system feeds in updated rules and practice tests automatically.

Image 1. UML Use Case Diagram: DriverPass System



UML Activity Diagrams

Actually, I picked up two use cases to break down step by step scheduling a driving appointment since that is the heart of the business and resetting a password, it's something every user type will need at some point.

Scheduling a Driving Appointment: customer picks a day and time, the system checks whether a car and driver are really free for that slot. If so, then walks them through payment before locking in the reservation and sending a confirmation. If the slot is not open or card is declined, then the customer gets a precise message and a chance to try again instead of the booking quietly failing.

Activity Diagram Below: Schedule Driving Appointment

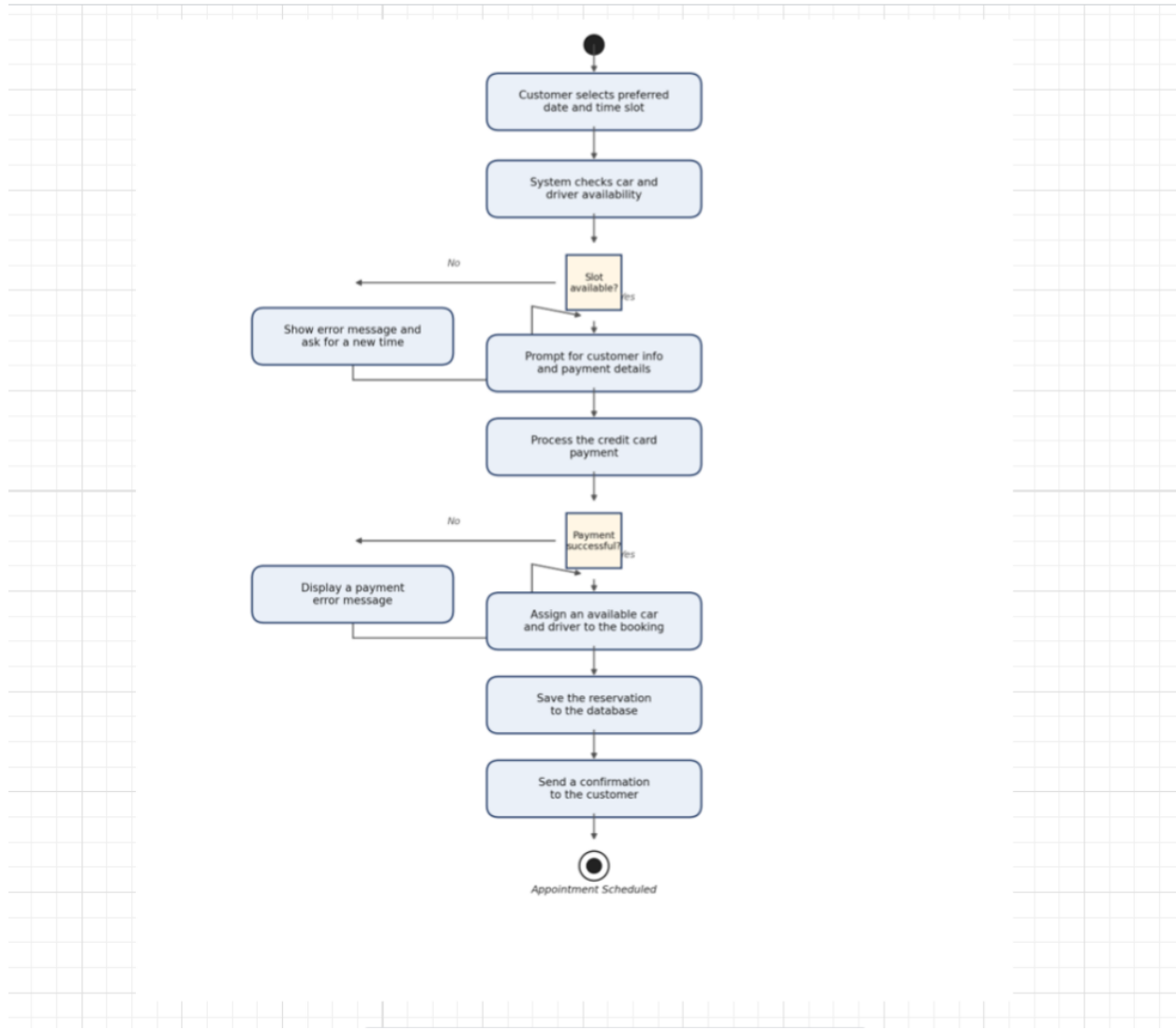


Image 2. Activity Diagram Above: Schedule Driving Appointment

Resetting Password: this covers what happens when someone forgets their login credentials.

Then, they ask for a reset, the system checks that the email is tied to an account for real. If it is, then it sends a secure link they can use to set a new password. If the email is not recognized, they

just get a friendly error instead of any hint about which emails exist in the system which matters for security purposes.

Activity Diagram Below: Reset Password

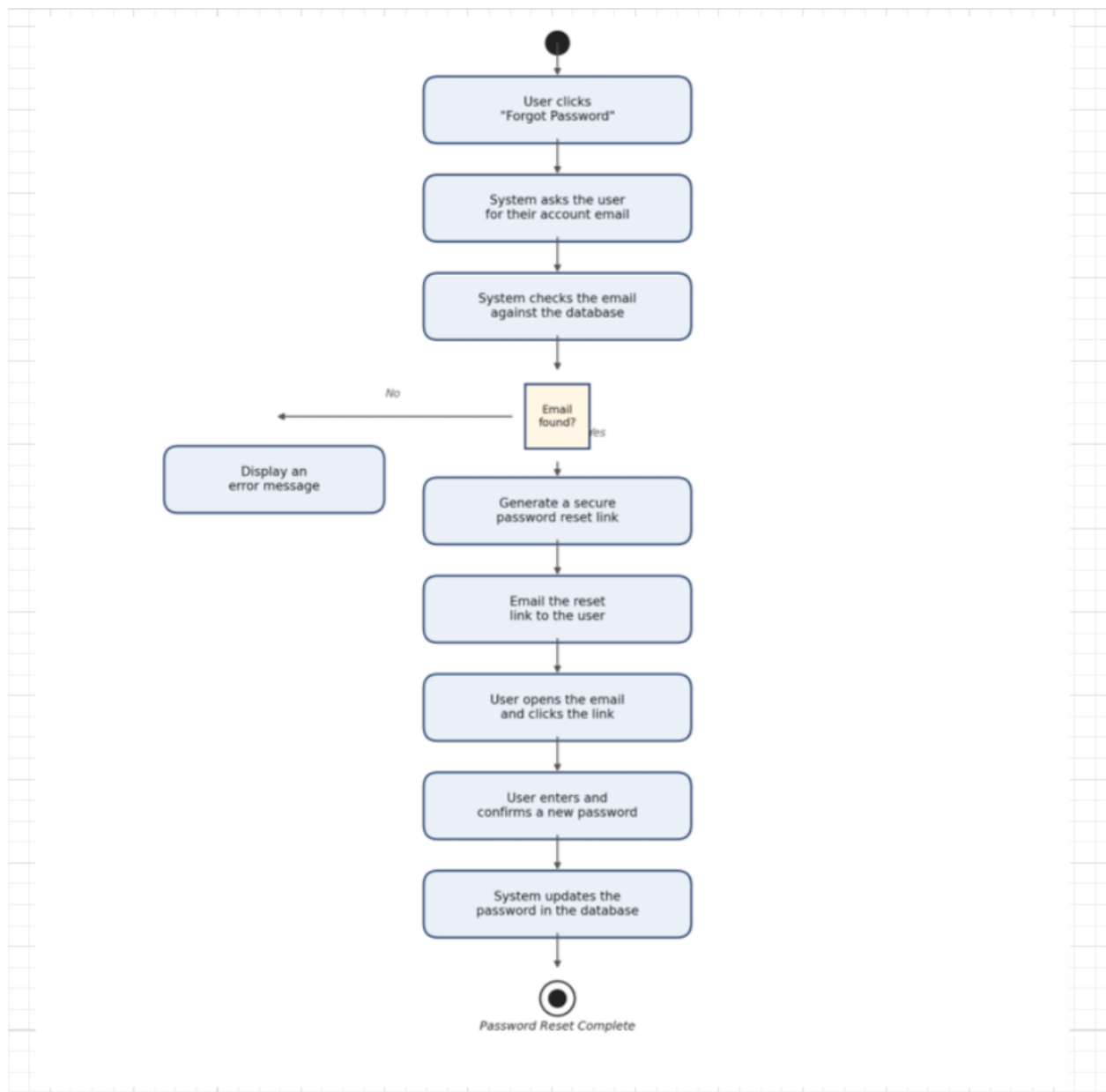


Image 3: Activity Diagram Above: Reset Password

UML Sequence Diagram

This diagram follows the “Schedule a Driving Appointment” use case again., but this time it shows the back and forth between the real pieces of the system: the customer, the website, the booking logic, and the database. It is really a play by play of the messages that get passed around behind the scenes by checking availability, saving the booking, and sending back a confirmation. So, the developers know exactly what needs to talk to what.

Sequence Diagram Below: Schedule Driving Appointment

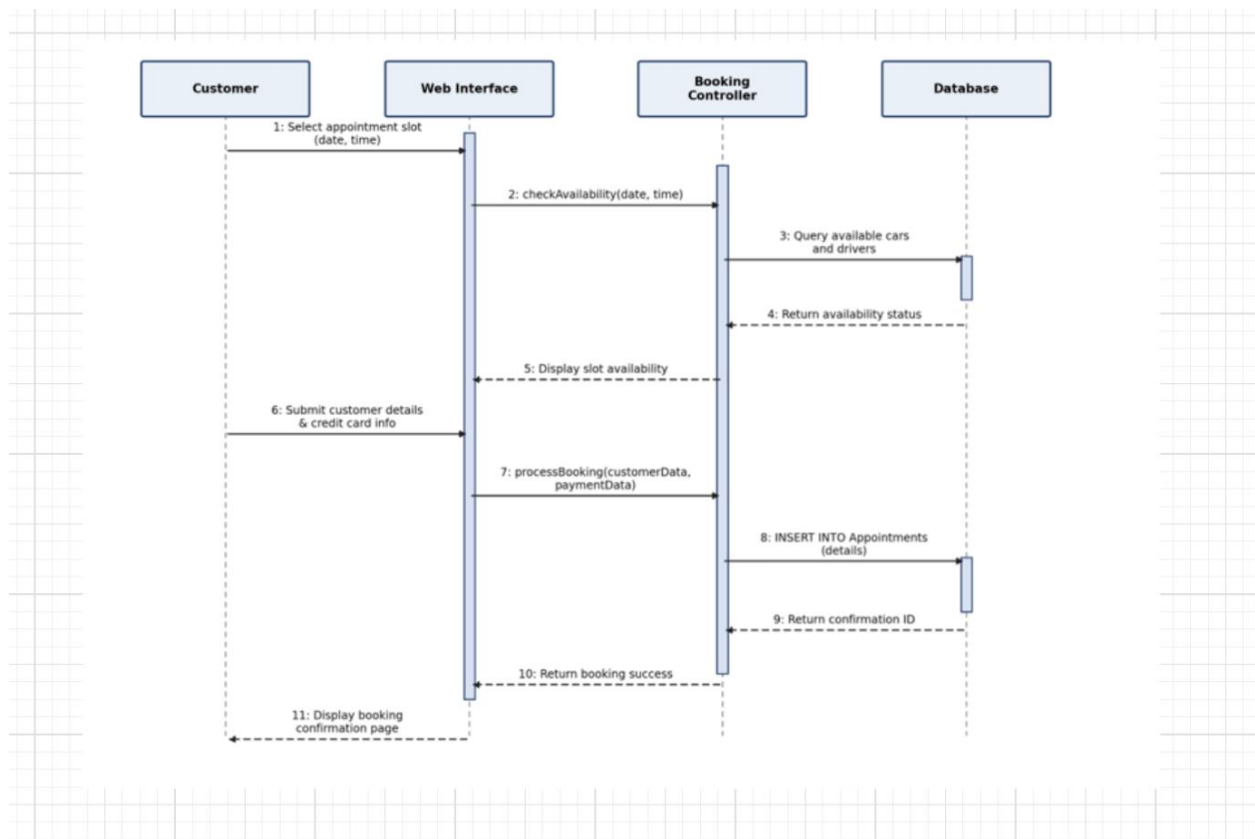


Image 4. UML Sequence Diagram Above: Schedule Driving Appointment

UML Class Diagram

This diagram lays out the main “things” the system needs to keep track of and how they really connect. A Customer schedules Appointments. Each Appointment gets matched with one Driver and one Car. Customers sign up for a Package the six/eight/twelve hour options Lian described, and every meaningful action gets written to an AuditLog so there is always a record of who did what by which covers the tracking and accountability piece Liam asked for.

Class Diagram Below: DriverPass System

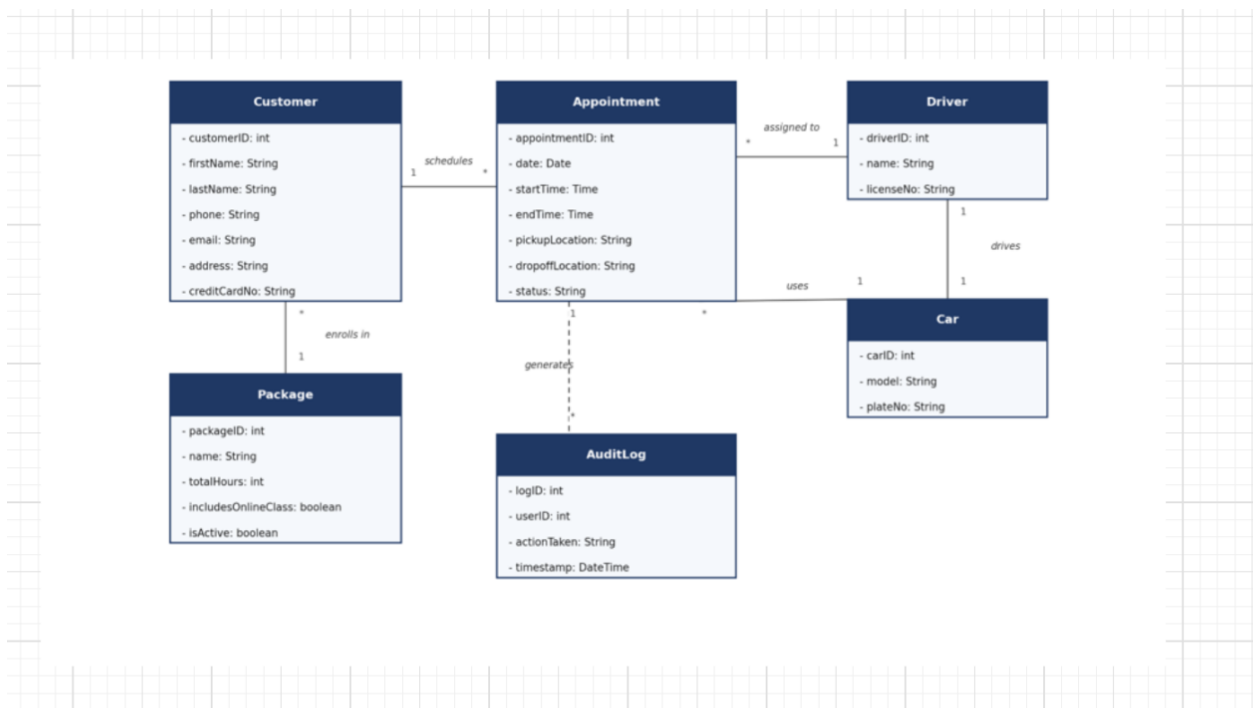


Image 5 UML Class Diagram Above: DriverPass System

Technical Requirements

The system will really need to run, in simple terms, based on everything DriverPass told us they needed:

Hardware

- A server to run the website and the database: nothing fancy, just a strong dual-core machine with 16 GB of RAM and superfast SSD drive so pages load rapidly, even when a lot of customers are booking at once.
- Customers and staff need a steady computer, tablet, or phone with a web browser: no special equipment or app install required.
- Drivers get a simple mobile friendly page to check their schedule and jot down lesson notes, so they don't need a laptop in the car.

Software

- The server runs on standard, reliable software Linux or Windows Server with a normal web server like Apache or Nginx handling traffic.
- All of the customer, appointment, and driver information lives in a regular relational database like MySQL or PostgreSQL: nothing exotic, just something proven and easy to maintain.
- Website itself is built with daily web tools HTML, CSS, JavaScript on the front end, and a standard backend framework such as Node.js or Django to handle the logic behind bookings, accounts, and payments.

Outside Connections and Tools

- The diagrams in my document were built in Lucidchart, where the dev team should keep using going forward so everything stays steady.
- A secure, PCI compliant payment processor like Stripe or PayPal handles the credit card charges, so DriverPass never has to store raw card numbers themselves; that is a big liability off their plate.
- A live connection to the DMV feeds in updated rules, policies, and practice questions automatically, the training content never goes stale.

Security and Infrastructure

- The entire system is hosted in the cloud like Azure or AWS, which is exactly what Liam asked for: he can check things from anywhere, on any device, without worrying about servers going down or losing data.
- Everything is backed up automatically off-site, so nobody at DriverPass has to think about backups; it just happens.
- Every page uses HTTPS SSL/TLS, so credit card numbers and personal details are encrypted whenever they travel over the internet.
- Each type of user customer, secretary, driver, IT officer, owner only sees and can do what makes sense for their role, this is what let Ian, as the IT officer, reset accounts and shut off access for someone who is left the company without giving that same power to everyone.
- Every cancellation, booking, and change gets logged automatically with who did it and when, so if something ever needs to be looked into, there's always a precise paper trail by covering the accountability piece Liam brought up in our interview.

###Reflection

Generally, I was working for a client called DriverPass. Their whole thing is helping people who keep failing their driving test actually pass it. They wanted me to build them a system where customers could sign up, take some online classes, do practice tests, and book real in-person driving lessons with a real instructor. It's not just a website but it is really the backend for their whole business-like booking lessons, matching people with cars and drivers, tracking packages, etc. The main point of contact was Liam (the owner) and Ian (IT guy). I think I did a solid job translating what the client was saying into real requirements. Therefore, they did not come to me saying "Liam wants to know who canceled a reservation" and I had to turn that into real functional requirements and a real system design. So, I feel like I was good at catching the small details they mentioned as an afterthought and making sure those actually showed up in the design instead of getting lost.

I would most likely go back and clean up my diagrams a bit more like my activity diagrams and class diagram could have been more consistent with each other. When you are moving fast among the use case diagram, activity diagrams, and class diagram it is easy for things to drift a little, so next time I would slow down and double check that each requirement really traces all the diagrams. Also, I basically kept going back to “who is really going to use this and what to do they need to do.” Customers needed something easy like book a lesson, pick a package, see their progress, etc., but Liam and Ian needed totally different things like admin access, activity logs, and the ability to turn packages on or off without any hesitation or error by the system. So, I had to design the system with separate roles and permissions baked in from the start, not bolted on later. This part matters so much because if you don’t really think on the different users, you might end up building something that looks fine on paper but does not really fit how people will use it daily.

My approach is generally talk to the client like a normal person first, don’t jump into diagrams right away. I like to try to pull out the real requirements from what they are saying before I even open a design tool. Then, I go functional versus nonfunctional, then use case diagrams to see the big image, after that activity diagrams to get into the weeds of each flow. After that, the class diagram once I really understand the data. So, moving forward I would most likely lean much more on going back and forth among the requirements document and the diagrams. So, nothing can be missed for sure.